

```

from google.colab import drive
import zipfile
import os
import shutil

drive.mount('/content/drive')

# Dosya Yolları
ZIP_PATH = '/content/drive/MyDrive/dataset.zip'
LOCAL_EXTRACT = '/content/dataset_working_v2'

# Temiz bir başlangıç için eski klasörü silip yeniden açıyoruz
if os.path.exists(LOCAL_EXTRACT):
    shutil.rmtree(LOCAL_EXTRACT)

print("Zip dosyası çıkartılıyor, lütfen bekleyin...")
with zipfile.ZipFile(ZIP_PATH, 'r') as zip_ref:
    zip_ref.extractall(LOCAL_EXTRACT)

# Klasör yapısını kontrol et (Gerekirse yolu güncellemek için)
print(f"Çıkartılan dosyalar: {os.listdir(LOCAL_EXTRACT)}")

```

```

Mounted at /content/drive
Zip dosyası çıkartılıyor, lütfen bekleyin...
Çıkartılan dosyalar: ['dataset']

```

```

import librosa
import numpy as np
import pandas as pd
import torch
from transformers import BertTokenizer, BertModel
from tqdm import tqdm
import os
import glob

# Cihaz ve Model Ayarları
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
bert_model = BertModel.from_pretrained('bert-base-uncased').to(device).eval()

# CSV Yolu
csv_path = os.path.join(LOCAL_EXTRACT, 'dataset', 'metadata_v1_en.csv')

# --- KRİTİK DÜZELTME: Dosya haritalama ---
print("Klasördeki tüm ses dosyaları taranıyor, bu işlem bir kez yapılır...")
# Tüm wav dosyalarını alt klasörler dahil bulur ve bir sözlüğe atar
audio_files_map = {os.path.basename(f): f for f in glob.glob(os.path.join(LOCAL_EXTRACT, '**', '*.wav'), recursive=True)}
print(f"Toplam {len(audio_files_map)} adet ses dosyası indekslendi.")

def get_features(text, audio_filename):
    # Dosya ismini sözlükten bul
    audio_path = audio_files_map.get(audio_filename)

    if audio_path is None or not os.path.exists(audio_path):
        raise FileNotFoundError(f"Dosya sistemde fiziksel olarak yok: {audio_filename}")

    # 1. MFCC (40 Katsayı)
    y, sr = librosa.load(audio_path, duration=3)
    mfcc = np.mean(librosa.feature.mfcc(y=y, sr=sr, n_mfcc=40).T, axis=0)

    # 2. BERT Embeddings (768)
    inputs = tokenizer(str(text), return_tensors='pt', truncation=True, padding=True, max_length=128).to(device)
    with torch.no_grad():
        outputs = bert_model(**inputs)
    bert_emb = outputs.last_hidden_state[:, 0, :].cpu().numpy().flatten()

    return np.concatenate([bert_emb, mfcc])

# CSV OKUMA
try:
    df = pd.read_csv(csv_path, sep=';', on_bad_lines='warn')
    # Sütun isimlerini temizle
    df.columns = [c.strip() for c in df.columns]
    print(f"CSV başarıyla yüklendi. Satır: {len(df)}")
except Exception as e:
    print(f"CSV okuma hatası: {e}")

X = []
y = []
label_map = {'negative': 0, 'neutral': 1, 'positive': 2}

print("Multimodal özellikler (BERT + MFCC) çıkarılıyor...")
for i, row in tqdm(df.iterrows(), total=len(df)):
    try:
        # Sizin CSV'nizdeki gerçek isimler: file_name, transcription, label
        text_data = row['transcription']
        audio_name = row['file_name']
        sentiment_label = str(row['label']).lower().strip()

        feat = get_features(text_data, audio_name)

```

```

    feat = get_features(text_data, audio_name)
    X.append(feat)
    y.append(label_map[sentiment_label])

except Exception as e:
    if i < 5: # Sadece ilk 5 hatayı göster ki kalabalık yapmasın
        print(f"\nSatır {i} hatası: {e}")
    continue

X = np.array(X)
y = np.array(y)
print(f"\nÖzellik çıkarımı tamamlandı. Başarıyla işlenen örnek sayısı: {len(X)}")

```

```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret 'HF_TOKEN' does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret i
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher
tokenizer_config.json: 100%          48.0/48.0 [00:00<00:00, 5.75kB/s]

vocab.txt:      232k/? [00:00<00:00, 4.12MB/s]

tokenizer.json:   466k/? [00:00<00:00, 15.2MB/s]

config.json: 100%          570/570 [00:00<00:00, 73.3kB/s]

model.safetensors: 100%          440M/440M [00:02<00:00, 551MB/s]

Loading weights: 100%          199/199 [00:00<00:00, 936.43it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: bert-base-uncased
Key | Status | |
-----+-----+
cls.predictions.bias | UNEXPECTED | |
cls.predictions.transform.LayerNorm.bias | UNEXPECTED | |
cls.seq_relationship.weight | UNEXPECTED | |
cls.predictions.transform.dense.weight | UNEXPECTED | |
cls.seq_relationship.bias | UNEXPECTED | |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED | |
cls.predictions.transform.dense.bias | UNEXPECTED | |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
Klasördeki tüm ses dosyaları taranıyor, bu işlem bir kez yapılır...
Toplam 8115 adet ses dosyası indekslendi.
CSV başarıyla yüklendi. Satır: 8107
Multimodal özellikler (BERT + MFCC) çıkarılıyor...
100%|██████████| 8107/8107 [03:37<00:00, 37.27it/s]
Özellik çıkarımı tamamlandı. Başarıyla işlenen örnek sayısı: 8107

```

```

import time
from sklearn.model_selection import cross_val_predict, KFold, cross_validate
from sklearn.metrics import classification_report, mean_squared_error
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from scipy import stats
import numpy as np
import os
import datetime

# Saniyeyi Saat:Dakika:Saniye formatına çevirir
def format_time(seconds):
    h = int(seconds // 3600)
    m = int((seconds % 3600) // 60)
    s = int(seconds % 60)
    return f"{h:02d}h {m:02d}m {s:02d}s"

# 1. Hazırlık
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
kf = KFold(n_splits=10, shuffle=True, random_state=42)
label_names = ['negative', 'neutral', 'positive']

models = {
    "Logistic Regression": LogisticRegression(max_iter=2000, class_weight='balanced'),
    "Random Forest": RandomForestClassifier(n_estimators=100, class_weight='balanced', random_state=42),
    "Gaussian NB": GaussianNB(),
    "Linear SVM": SVC(kernel='linear', class_weight='balanced', probability=True),
    "K-Nearest Neighbors": KNeighborsClassifier(n_neighbors=5)
}

save_dir = '/content/drive/MyDrive/May_Revision_Package_V2'
if not os.path.exists(save_dir): os.makedirs(save_dir)
ts = datetime.datetime.now().strftime("%Y%m%d_%H%M")

file_class_reports = os.path.join(save_dir, f"TableA_Classification_Reports_{ts}.txt")
file_deep_metrics = os.path.join(save_dir, f"TableB_Deep_Analysis_{ts}.txt")

```

```

model_scores_dict = {}
deep_metrics_log = []
class_reports_log = []

print("Kapsamlı analiz (Ablasyon + Significance + Time) başladı...")

for name, model in models.items():
    print(f"İşleniyor: {name}...")
    start_time = time.time()

    # --- 1. Temel Tahmin ve Zaman Ölçümü ---
    y_pred = cross_val_predict(model, X_scaled, y, cv=kf, n_jobs=-1)
    training_duration = time.time() - start_time

    # --- 2. K-Fold Doğruluk Verileri ---
    cv_full = cross_validate(model, X_scaled, y, cv=kf, n_jobs=-1, scoring='accuracy')
    model_scores_dict[name] = cv_full['test_score']

    # --- 3. Ablasyon Analizi (Tekil Özellik Katkısı) ---
    cv_bert = cross_validate(model, X_scaled[:, :768], y, cv=kf, n_jobs=-1)['test_score'].mean()
    cv_mfcc = cross_validate(model, X_scaled[:, 768:], y, cv=kf, n_jobs=-1)['test_score'].mean()

    # --- 4. Raporlama ---
    # Tablo A (Sınıf Bazlı)
    report_text = classification_report(y, y_pred, target_names=label_names, digits=4)
    class_reports_log.append(f"ALGORITHM: {name}\n{report_text}\n" + "="*60 + "\n")

    # Tablo B (Derin Metrikler)
    rmse_val = np.sqrt(mean_squared_error(y, y_pred))
    deep_metrics_log.append({
        'name': name,
        'acc': cv_full['test_score'].mean(),
        'std': cv_full['test_score'].std(),
        'rmse': rmse_val,
        'time': format_time(training_duration),
        'bert_acc': cv_bert,
        'mfcc_acc': cv_mfcc
    })

# --- 5. Anlamlılık Testi (Paired t-test) ---
best_algo = max(model_scores_dict, key=lambda k: model_scores_dict[k].mean())
significance_text = f"\nSTATISTICAL SIGNIFICANCE (Reference: {best_algo})\n"
for name in models:
    if name != best_algo:
        t_stat, p_val = stats.ttest_rel(model_scores_dict[best_algo], model_scores_dict[name])
        significance_text += f"- {best_algo} vs {name}: p-value = {p_val:.6f} {'(Significant)' if p_val < 0.05 else '(Not Significant)'}\n"

# --- 6. Kayıt ---
with open(file_class_reports, 'w') as f:
    f.write(f"TABLE A: CLASSIFICATION REPORTS (Multimodal)\nDate: {ts}\n\n")
    f.write("\n".join(class_reports_log))

with open(file_deep_metrics, 'w') as f:
    f.write(f"TABLE B: RELIABILITY, ABLATION AND EFFICIENCY\nDate: {ts}\n\n")
    f.write("-" * 95 + "\n")
    f.write(f"{'Algorithm':<20} | {'Acc.':<7} | {'Std.':<7} | {'RMSE':<7} | {'Training Time':<12} | {'BERT':<7} | {'MFCC':<7}\n")
    f.write("-" * 95 + "\n")
    for m in deep_metrics_log:
        f.write(f"{m['name']:<20} | {m['acc']:.4f} | {m['std']:.4f} | {m['rmse']:.4f} | {m['time']:<12} | {m['bert_acc']:.4f} | {m['mfcc_acc']:.4f}\n")
    f.write(significance_text)

print(f"Bitti! Dosyalarınız Drive'da: \n1. {file_class_reports}\n2. {file_deep_metrics}")

```

Kapsamlı analiz (Ablasyon + Significance + Time) başladı...  
 İşleniyor: Logistic Regression...  
 İşleniyor: Random Forest...  
 İşleniyor: Gaussian NB...  
 İşleniyor: Linear SVM...  
 İşleniyor: K-Nearest Neighbors...  
 Bitti! Dosyalarınız Drive'da:  
 1. /content/drive/MyDrive/May\_Revision\_Package\_V2/TableA\_Classification\_Reports\_20260515\_1220.txt  
 2. /content/drive/MyDrive/May\_Revision\_Package\_V2/TableB\_Deep\_Analysis\_20260515\_1220.txt

